

# Weekly Research Progress Report

---

**Project:** Dual-Arm Robot Tomato Cutting (DOM) — Paper 1 Execution

**Authors:** Ahmed & Shahd

**Lab:** DEX-ROB Lab, Tianjin University

**Advisor:** Prof. Shan An

**Date:** September 19, 2026

---

## 1. Executive Summary

---

This week, we focused on finalizing our software toolchain, setting up our Linux development environment, installing and testing the simulation pipeline, and benchmarking system performance on our hardware platform.

We have successfully installed and verified **Isaac Sim** and **Isaac Lab**, established our complete Linux development stack, and evaluated the simulator efficiency between **Isaac Lab** and **MuJoCo + Isaac Sim**. During performance benchmarking, we identified a critical RAM memory bottleneck on our laptop when running parallel simulation environments, leading us to recommend a RAM upgrade to 32 GB.

---

## 2. Recommended Software Stack & Simulator Evaluation

---

# Proposed 7-Layer Software Architecture

To execute Paper 1 (*Adaptive Slicing of Deformable Objects via Traditional RL & Phase-Aware Impedance Control*), we designed a 7-layer modular software architecture:

|                                           |
|-------------------------------------------|
| Layer 7: Monitoring & Logging             |
| Layer 6: RL Training Framework            |
| Layer 5: Deep Learning & Policy Network   |
| Layer 4: Perception / Vision (Real Robot) |
| Layer 3: Simulation Environment           |
| Layer 2: Robot Middleware & Control       |
| Layer 1: System Foundation                |

## Simulator Comparison: Isaac Lab vs. MuJoCo + Isaac Sim

We performed a detailed architectural evaluation comparing **Isaac Lab (Unified Framework)** against a **MuJoCo + Isaac Sim (Hybrid Pipeline)** for our Paper 1 traditional RL training requirements:

| Feature / Dimension | ■ Isaac Lab (Selected)                                                    | ● MuJoCo + Isaac Sim (Hybrid)                                                     |
|---------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Architecture        | <b>Unified GPU Pipeline</b><br>(PhysX 5 + RTX Render + RL in one process) | <b>Two Separate Processes</b> (MuJoCo CPU physics synced to Isaac Sim GPU render) |

|                                |                                                                       |                                                                       |
|--------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>RL Training Speed</b>       | ☆ <b>Extremely Fast</b><br>(1,000–4,096 parallel environments on GPU) | ⚠ <b>Moderate/Slow</b> (IPC sync & PCIe CPU-GPU transfer bottlenecks) |
| <b>Contact Dynamics</b>        | Good for rigid/deformable approximation via PhysX 5                   | Gold standard for analytical rigid body contact                       |
| <b>Framework Integration</b>   | Native PyTorch & SkRL support                                         | Requires custom gym wrappers & IPC bridges                            |
| <b>Setup &amp; Maintenance</b> | Single unified NVIDIA stack                                           | Higher complexity (maintaining two engines)                           |

**Conclusion & Recommendation:**

For Paper 1, where traditional Reinforcement Learning (PPO/SAC) requires millions of environment steps, **Isaac Lab** is decisively superior due to its end-to-end GPU parallelization (10×–100× faster training throughput). We will use Isaac Lab as our primary simulation environment.

---

### 3. Development Environment Setup

---

We have completed the full Linux system configuration:

1. **Linux OS:** Configured Ubuntu 22.04 LTS via WSL2 with full NVIDIA GPU passthrough and CUDA 12/13 support.
  2. **Package Management:** Configured Conda/Mamba virtual environments (`isaacsim` for simulation and `ros2` for hardware control) to prevent library conflicts.
  3. **Toolchains:** Installed PyTorch, CUDA toolkits, SkRL, Weights & Biases (WandB), Hydra config management, and ROS 2 middleware dependencies.
-

## 4. Isaac Sim & Isaac Lab Installation Status

---

We have successfully installed and verified both **Isaac Sim** and **Isaac Lab** on our development system:

- Built and verified the Isaac Lab core repository (`./isaacclab.sh --install`).
  - Verified GPU acceleration, USD stage rendering, and PhysX 5 physics engine initialization.
  - Tested baseline Gymnasium robot environments (`Isaac-Cartpole-v0`, `Isaac-Reach-Franka-v0`).
- 

## 5. Hardware Benchmarking & RAM Upgrade Request

---

During our stress-testing and benchmarking of Isaac Sim & Isaac Lab with parallel environments, we identified a critical hardware performance bottleneck:

### Current Hardware Specifications

- **CPU:** Intel Core i7-14650HX (16 Cores / 24 Threads)
- **GPU:** NVIDIA GeForce RTX 5060 Laptop GPU (8 GB VRAM)
- **RAM:** 16.0 GB DDR5

### Benchmarking Observation & Technical Bottleneck

- When running Isaac Sim along with PyTorch, WSL2 system overhead, and multiple parallel simulation instances, **RAM usage rapidly reaches 95%–98%** (~15.2 GB / 15.6 GB).
- This memory saturation forces Windows and WSL2 to page memory onto disk (SSD swap), resulting in severe simulation frame drops, system unresponsiveness, and occasional Out-Of-

Memory (OOM) process crashes.

- While the CPU and GPU (RTX 5060 with 8GB VRAM) are performing excellently, the **16 GB RAM is the single limiting factor** for scalable training.

## Hardware Upgrade Recommendation

- Our laptop hardware natively supports up to **32 GB DDR5 RAM** (2 × 16 GB configuration).
  - We strongly recommend upgrading the system memory from **16 GB to 32 GB DDR5**.
  - **Impact of Upgrade:** Upgrading to 32 GB RAM will completely eliminate disk swapping, allow allocation of 24 GB RAM to WSL2, and enable stable, uninterrupted RL policy training across 1,000+ parallel environments.
- 

## 6. Next Week's Plan

---

1. **Formalize MDP Implementation for Paper 1:** Implement the mathematical State (\$\$\$), Action (\$A\$), and Reward (\$R\$) functions for tomato cutting in Python/Isaac Lab code.
  2. **Build Custom Isaac Lab Tomato Cutting Environment:** Develop `DirectRLEnv` / `ManagerBasedRLEnv` for the dual-arm DOM setup with tomato mesh asset loading and TacBlade force sensor readings.
  3. **Baseline Policy Training:** Initiate baseline PPO policy training runs in Isaac Lab and log metrics via WandB.
- 

*Submitted by: Ahmed & Shahd  
DEX-ROB Lab, Tianjin University*